

CDS View (Core Data Services View)

- SAP의 새로운 인메모리 DB인 HANA DB를 위한 SQL 기반의 뷰
 - 인메모리 DB : 디스크 대신 메인 메모리(RAM)에 데이터를 저장하는 데이터베이스
- CDS View는 **“데이터를 어떻게 가져올지가 아니라, 이 데이터가 무엇인지(의미)를 정의해 두는 설계도”**
 - 또한 이 데이터가 무슨 의미를 가지는지도 **Annotation**이라는 주석을 사용해서 정의하고 있음.
 - 각 컬럼이 무엇을 의미하는지, 어떤 테이블과 어떤 관계인지(1:n, 부모-자식), 권한 체크를 해야하는지 등...
 - Fiori UI5 화면이나 분석 관련해서 이 데이터가 어떻게 사용되거나 출력되어야 하는지 등..
 - SQL View는 이런 거 못함
- CDS View는 **OData 서비스, Fiori UI 등 여러 곳에서 재사용될 목적**으로 만들어졌음.
 - 추후 배울 CDS View와 1:1 관계로 연결될 **Behavior Definition**을 쓰면 데이터 수정/삭제/검증도 가능함
 - Maintenance View와 SM30의 역할을 후속으로 이어받는다고 이해하면 쉬움

CDS View

```
define view ZCL3_20_08_DDL
with parameters
  p_aufnr : aufnr
as select
  key a.aufnr,
  key b.posnr,
    a.gltrp,
    a.gamng,
    a.gmein,
    b.matnr,
    c.maktx,
    b.bwtar

  from          afko as a
  inner join    afpo as b on a.aufnr = b.aufnr
  left outer join makt as c on c.matnr = b.matnr
                                and c.spras = $session.system_language
  where a.aufnr = $parameters.p_aufnr;
```

```
@AbapCatalog.sqlViewName: 'ZCL3_CDS_20_08'
@AbapCatalog.compiler.compareFilter: true
@AccessControl.authorizationCheck: #NOT_REQUIRED
@EndUserText.label: 'ZCL3_20_08_DDL'
@Metadata.ignorePropagatedAnnotations: true
```

AMDP / AMDF (ABAP Managed Data ...)

- 원래 우리가 테이블을 처리하던 방식은 꽤나 비효율적이었다. (속도도 느리고 DB는 놀고 있음)
 - DB에서 데이터 조회하고, ABAP 서버로 대량 데이터 전송하며, LOOP / 계산은 ABAP에서 처리
- 아까 이야기한 HANA DB는 계산에 최적화된 인메모리 DB이므로 DB에서 계산하는게 빠름 (병렬 처리)
 - ABAP 리포트 프로그램에서 작성할 코드를 DB 구문으로 보내고, 대량 데이터를 계산하는건 이게 낫다
- 즉 AMDP는 속도를 위해서 ABAP 클래스 안에서 DB에서 실행될 SQL Script를 작성하는 방식이다.
 - 이걸 ABAP 코드처럼 관리하고, 실제 실행 및 계산은 HANA DB가 하는 것
 - IF_AMDP_MARKER_HDB 라는 인터페이스를 꼭 구현(implements) 해야 함.
 - 이 인터페이스는 "이 클래스는 AMDP를 포함하고 있습니다"라는 표식(Marker) 역할을 함.
- AMDF는 ABAP 클래스 내에서 Function Module 대신 DB에서 실행되는 함수를 만드는 것
 - 특정 테이블을 조회하고, 그 결과를 AMDP에서 사용하는 등 여러가지 일을 할 수 있음, 무조건 Read-Only

AMDF - AMDP

```
METHOD calc_avg
  BY DATABASE FUNCTION
  FOR HDB
  LANGUAGE SQLSCRIPT
  OPTIONS READ-ONLY.

  RETURN :iv_sum / :iv_cnt;

ENDMETHOD.
```

```
METHOD get_avg_by_bukrs
  BY DATABASE PROCEDURE
  FOR HDB
  LANGUAGE SQLSCRIPT
  USING bsis.

  et_result =
    SELECT
      bukrs,
      zcl_calc_amdf⇒calc_avg(
        SUM( wrbtr ),
        COUNT( * )
      ) AS avg_amt
    FROM bsis
    WHERE mandt = session_context( 'CLIENT' )
    GROUP BY bukrs;

ENDMETHOD.
```

ADBC (ABAP Database Connectivity)

- ABAP에서 SQL 문자열을 직접 DB로 보내기 위한 클래스 기반 API
 - 즉 메인 데이터베이스가 아닌, 서브 데이터베이스에 직접 접근하기 위한 방법
 - 각 데이터베이스의 언어를 그대로 문자열로 담아 보내는 것

ADBC 주요 Class 및 역할

Class	역할
CL_SQL_CONNECTION	• 연결생성 및 관리 : 기본DB외 외부(Secondary)DB와의 연결을 설정한다. • 트랜잭션 제어 : SQL문을 통한 COMMIT, ROLLBACK 대신 이 클래스의 메소드를 사용해서 처리해야함. • 핵심 메소드 : GET_CONNECTION, COMMIT, ROLLBACK, CLOSE, GET_DBMS
CL_SQL_STATEMENT	• SQL문장을 DB에 전달하고 실제로 명령을 실행하는 역할을 한다. • 주요역할 : SELECT, INSERT, UPDATE, DELETE와 같은 Query 를 날리거나 Procedure를 호출한다. • 핵심 메소드 : 1. EXECUTE_QUERY : SELECT 쿼리를 실행하고 결과로 CL_SQL_RESULT_SET 객체를 반환. • 2. EXECUTE_UPDATE : UPDATE, INSERT, DELETE를 수행하며 변경된 ROW수를 반환. • 3. EXECUTE_DDL : 테이블의 CREATE, DROP같은 스키마변경 작업을 함. • 4. EXECUTE_PROCEDURE : Stored procedure를 호출할 때 사용.
CL_SQL_RESULT_SET	• 결과보관소 역할 : SELECT 실행 후 돌아온 데이터 결과물을 ABAP변수나 ITAB에 옮겨주는 역할을 한다. • 주요역할 : 쿼리 실행 결과로 넘어온 데이터 패키지를 순차적으로 읽어 들인다. • 핵심 메소드 : 1. SET_PARAM : 조회결과를 담을 변수에 연결(Binding)한다. • 2. SET_PARAM_STRUCT : 조회결과를 담을 Work area에 연결(Binding)한다. • 3. SET_PARAM_TABLE : 조회결과를 담을 Internal table에 연결(Binding)한다. • 4. NEXT_PACKAGE : 결과 데이터를 실제 Internal table로 가져온다. • 5. NEXT : 결과 데이터를 변수 또는 Work area로 가져온다. • 6. CLOSE : 쿼리 수행이 끝나면 데이터베이스 리소스를 해제한다. 필수 메소드!!
요약	
1. CL_SQL_CONNECTION으로 DB 연결을 잡는다. 2. CL_SQL_STATEMENT로 SQL 쿼리를 실행한다. 3. CL_SQL_RESULT_SET으로 결과 데이터를 받아 Internal table 에 담는다.	

ADBC (ABAP Database Connectivity)

- 1. 외부 데이터베이스와 연결 `CL_SQL_CONNECTION` `get_connection()`
- 2. 외부 데이터베이스에 실행할 SQL문을 만듦
- 3. 만든 SQL문을 실행 `CL_SQL_STATEMENT` `execute_query()`
- 4. SQL문의 실행결과를 받아올 변수를 지정 (변수 OR Workarea OR Internal Table)
`CL_SQL_STATEMENT`
`set_param_table()`
`set_param()`
- 5. SQL문의 실행결과를 받아옴 `CL_SQL_STATEMENT` `next_package()` `next()`
- 6. 사용이 끝났으면 DB 리소스 정리 `CL_SQL_STATEMENT` `close()`

New Syntax

D75AW

Figure 69: Reading an Internal Table With The
Table Expression

Unit 4 Lesson 3

```
DATA: gt_flights TYPE TABLE OF sflight,  
      gs_flight  LIKE LINE OF gt_flights.  
  
READ TABLE gt_flights INTO gs_flight INDEX 1.  
  
READ TABLE gt_flights INTO gs_flight WITH KEY carrid = 'LH'  
                                         connid = '0400'.  
  
READ TABLE gt_flights INTO gs_flight  
                        WITH TABLE KEY fld  
                        COMPONENTS fldate = '20150401'.
```

Before

```
DATA: gt_flights TYPE TABLE OF sflight,  
      gs_flight  LIKE LINE OF gt_flights,  
      gv_seatsmax TYPE sflight-seatsmax.  
  
gs_flight = gt_flights[ 1 ].  
  
gs_flight = gt_flights[ carrid = 'LH' connid = '0400' ].  
  
gs_flight = gt_flights[ KEY fld COMPONENTS fldate = '20150401' ].  
  
gv_seatsmax = gt_flights[ 1 ]-seatsmax.
```

After

New Syntax

D75AW

Figure 59: Dynamic WHERE Conditions For
Internal Tables

Unit 4 Lesson 1

```
METHODS get_flights
    IMPORTING iv_carrid TYPE sflight-carrid
              iv_connid TYPE sflight-connid OPTIONAL
    EXPORTING et_flights TYPE ty_flights.
```

```
DATA: lv_where TYPE string,
      ls_flight TYPE sflight.

lv_where = |carrid = { iv_carrid }|.
IF iv_connid IS SUPPLIED.
    lv_where = |{ lv_where } AND connid = { iv_connid }|
ENDIF.

LOOP AT mt_flights INTO ls_flight WHERE (lv_where).
    INSERT ls_flight INTO TABLE et_flights.
ENDLOOP.
```


New Syntax

D75AW

Figure 43: Creating Objects With NEW()

Unit 3 Lesson 4

```
DATA: go_custom TYPE REF TO cl_gui_custom_container,  
      go_alv TYPE REF TO cl_gui_alv_grid.
```

```
CREATE OBJECT go_custom  
  EXPORTING container_name = 'CUSTOM100'.
```

```
CREATE OBJECT go_alv  
  EXPORTING i_parent = go_custom.
```

Before

```
DATA: go_custom TYPE REF TO cl_gui_custom_container,  
      go_alv TYPE REF TO cl_gui_alv_grid.
```

```
go_custom = NEW #( container_name = 'CUSTOM100' ).
```

```
go_alv = NEW #( i_parent = go_custom ).
```

After

New Syntax

D75AW

Figure 48: Creating Variables With VALUE()

Unit 3 Lesson 5

```
DATA: gt_range TYPE RANGE OF sflight-carrid,  
      gs_range LIKE LINE OF gt_range.
```

```
gs_range-sign    = 'I'.  
gs_range-option  = 'BT'.  
gs_range-low     = 'AA'.  
gs_range-high    = 'DL'.
```

```
APPEND gs_range TO gt_range.
```

```
class=>method( it_range = gt_range ).
```

Before

```
class=>method( it_range = VALUE #(  
    ( sign    = 'I'  
      option = 'BT'  
      low    = 'AA'  
      high   = 'DL'  
    )  
)  
).
```

After

New Syntax

D75AW

Figure 48: Creating Variables With VALUE()

Unit 3 Lesson 5

```
DATA: gt_range TYPE RANGE OF sflight-carrid,  
      gs_range LIKE LINE OF gt_range.
```

```
gs_range-sign    = 'I'.  
gs_range-option  = 'BT'.  
gs_range-low     = 'AA'.  
gs_range-high    = 'DL'.
```

```
APPEND gs_range TO gt_range.
```

```
class=>method( it_range = gt_range ).
```

Before

```
class=>method( it_range = VALUE #(  
    ( sign    = 'I'  
      option = 'BT'  
      low    = 'AA'  
      high   = 'DL'  
    )  
  )  
).
```

After